Recapitulation Test

Loops, Nested Loops, Patterns, Sequences and Series (Python)

Instructions

- Solve every problem in Python using only **for** loops, **while** loops, nested loops, conditionals and basic lists.
- Do **not** define any function or class. Do not use recursion or built-in shortcuts that replace the loop logic.
- Read input with **input()** and match the sample output format exactly, including spaces and line breaks.

Problem 1: Concentric Parity Square

Read an integer **n**. Build a square grid of side **2n - 1**. The value of each cell is **n** minus the distance from that cell to the nearest edge of the grid, so the outer ring holds **n** and the center holds **1**.

Print the value if it is **odd**. If it is **even**, print ***** instead. Print each row with no spaces between characters.

- Input: one integer n (1 <= n <= 9).
- Use nested loops only; do not store the grid in a list.

Sample Input 1	Sample Output 1
3	33333 3***3 3*1*3 3***3 33333

Sample Input 2	Sample Output 2
4	******* *33333* *3***3* *3*1*3* *3***3* *33333* *******

Problem 2: Pascal-Sierpinski Triangle

Read an integer **rows**. Print the first **rows** rows of Pascal's triangle, centered, with **#** for every **odd** entry and **.** for every **even** entry. Separate symbols in a row with a single space.

Compute each entry inside the inner loop using the relation **c = c * (i - j) // (j + 1)**, where **i** is the row number and **j** is the position in the row (both starting at 0). Do not store earlier rows and do not use **math.comb**.

- Input: one integer rows (1 <= rows <= 16).
- Leading spaces create the centered shape; there should be no trailing spaces.

Sample Input 1	Sample Output 1

4	`   #`
	`  # #`
	` # . #`
	`# # # #`

Sample Input 2	Sample Output 2

6	`     #`
	`    # #`
	`   # . #`
	`  # # # #`
	` # . . . #`
	`# # . . # #`

Problem 3: Digit-Square Chains

Read an integer **x**. Repeatedly replace **x** with the **sum of the squares of its digits**. Extract the digits with an inner **while** loop (using **%** and **//**); do not convert the number to a string.

Stop when **x** becomes **1**, or when **x** has already appeared earlier in the chain. Print the list of values visited (not including the value that caused the stop), the final value of **x**, and the number of replacements made.

- Input: one positive integer x.
- Use a list to remember the values already seen.

Sample Input 1	Sample Output 1

7	`Chain: [7, 49, 97, 130, 10]`
	`Final x: 1`
	`Replacements: 5`

Sample Input 2	Sample Output 2

2	`Chain: [2, 4, 16, 37, 58, 89, 145, 42, 20]`
	`Final x: 4`
	`Replacements: 9`

Problem 4: Filtered Fibonacci Accumulator

Read an integer **limit**. Generate the Fibonacci sequence **1, 2, 3, 5, 8, 13, ...**. **Ignore** every term that is divisible by 3, but keep generating the sequence past it.

Add each accepted term to a running sum. Stop as soon as the sum becomes **greater than limit**. Print the accepted terms, the final sum, how many terms were accepted, and the term that caused the stop.

- Input: one integer limit (limit >= 1).
- Use a **while** loop; the number of terms is not known in advance.

Sample Input 1	Sample Output 1

`30`	`Accepted terms: [1, 2, 5, 8, 13, 34]`
	`Sum: 63`
	`Count: 6`
	`Stopped at: 34`

Sample Input 2	Sample Output 2

`100`	`Accepted terms: [1, 2, 5, 8, 13, 34, 55]`
	`Sum: 118`
	`Count: 7`
	`Stopped at: 55`

Problem 5: Pythagorean Perimeter Hunt

Read two integers: **N** on the first line and **P** on the second. Using nested **for** loops over **1 <= a <= b < N**, find every Pythagorean triple **(a, b, c)** with $a^2 + b^2 = c^2$ and **a + b + c <= P**.

You may **not** use ****0.5**, **sqrt**, or **isqrt** to find **c**. Find it with a **while** loop instead. Print each triple on its own line as **a b c** in the order found, then print the total count.

- Input: integer N, then integer P, each on its own line.
- If no triple qualifies, print only the total (0).

Sample Input 1	Sample Output 1

`20`
`40`
	`3 4 5`
	`5 12 13`
	`6 8 10`
	`8 15 17`
	`9 12 15`
	`Total: 5`

Sample Input 2	Sample Output 2

`15`
`30`
	`3 4 5`
	`5 12 13`
	`6 8 10`
	`Total: 3`